Explicación Integral del Documento de Avance

Materia: Tecnologías Orientadas a Objetos

Integrantes: Pablo De Jesús Salazar Rioú (clave 350287) y Francisco Javier Silos Villegas (clave 333127)

Profesor Evaluador: Uriel Juárez Martínez

Fecha de Entrega: 16 de octubre del 2026

1. Visión General del Proyecto

El proyecto consiste en el desarrollo de un **Sistema de Gestión Cinematográfica y Venta de Boletos** con interfaz gráfica en Java Swing (`JFrame`). El sistema administra sucursales de cines, salas físicas, catálogos de películas, programación de funciones, asignación de butacas en tiempo real y ventas de boletos con procesamiento polimórfico de pagos.

> **Mensaje del Profesor en la Imagen:**
> *"El ejercicio del examen, el avance se entrega el viernes donde tendrán que venir a explicarme sus documentos y la prueba del desarrollo"*.
> Esta guía condensa la explicación completa y detallada de todo el documento: desde el diagrama UML hasta el esquema relacional de base de datos y la prueba visual.

2. Análisis Superdetallado del Diagrama UML (Página 2)

El diagrama de clases modela tanto la **capa gráfica / presentación** como la **capa de dominio / lógica de negocio**:

A. Capa de Presentación (GUI)

- `Principal` : Punto de entrada del sistema (`main(args : String[]) : void`). Instancia el objeto central `Sistema` y arranca la interfaz mediante `VentanaLogin` .
- `VentanaLogin` (hereda de `JFrame`):
  - Mantiene una referencia a `sistema : Sistema` .
  - `abrirVentanaAdmin(pinIngresado : String) : void` : Valida las credenciales del administrador y levanta el panel de gestión.
  - `abrirVentanaCliente() : void` : Otorga acceso directo al flujo de consulta y compra de boletos.
- `VentanaAdmin` (hereda de `JFrame`):
  - Mantiene referencias a `sistema : Sistema` y `adminActivo : Administrador` .
  - Métodos operativos: `registrarPelicula()` , `modificarPelicula()` , `eliminarPelicula()` , `agregarSala()` , `programarFuncion()` y `cerrarSesion()` .
- `VentanaCliente` (hereda de `JFrame`):
  - Mantiene referencias a `sistema : Sistema` y `clienteActivo : Cliente` .

- Métodos del flujo comercial: `mostrarCartelera()`, `seleccionarFuncion()`, `mostrarMatrizAsientos()`, `realizarCompra()` y `cerrarSesion()`.

B. Capa de Dominio y Lógica Central

- `Sistema` (Patrón Fachada / Controlador Principal):
  - Atributos de colección: `cines : List<Cine>`, `catalogoPeliculas : List<Pelicula>` y `boletosVendidos : List<Boleto>`.
  - Métodos clave: `inicia() : boolean`, CRUD de cines (`agregaCine`, `obtenerCine`, `eliminarCine`, `modificarCine`), CRUD del catálogo de películas (`agregarPelicula`, `modificarPelicula`, `eliminarPelicula`, `obtenerPelicula`), búsquedas especializadas (`buscarPeliculasPorActor`, `buscarPeliculasPorClasificacion`), programación y cancelación de funciones, y `registrarBoleto(boleto : Boleto)`.
- **Jerarquía de Usuarios (Herencia y Polimorfismo):**
  - `Usuario` (Clase Abstracta): Define `nombre : String` y el método `getNombre() : String`.
  - `Administrador` (subclase de `Usuario`): Añade `pinAcceso : String`, `validarPin(pinIngresado : String) : boolean` y métodos operativos que actúan sobre el sistema activo (`registraPelicula`, `modificaPelicula`, `programarFuncion`, etc.).
  - `Cliente` (subclase de `Usuario`): Implementa la transacción `comprarBoleto(funcion : Funcion, idSala : String, seleccion : List<Asiento>, metodoPago : MetodoDePago) : Boleto`.

C. Entidades del Negocio Cinematográfico

- `Cine`: Modela una sucursal con `nombreCine : String` y `salas : List<Sala>`. Gestiona sus propias salas (`agregaSala`, `obtenerSala`, `eliminaSala`, `modificaSala`) y permite programar y cancelar funciones delegadas en sus salas.
- `Sala`: Modela las dimensiones físicas y tarifa base: `idSala : String`, `filas : int`, `asientosPorFila : int`, `precioBase : double`, `asientos : List<Asiento>` y `programacion : List<Funcion>`. Cuenta con el método `generarAsientos(filas, asientoPorFila, tipo, precio)` para armar la cuadrícula.
- `Pelicula`: Encapsula la información técnica: `titulo`, `duracion`, `clasificacion`, `genero`, `director`, `actores : List<String>` y `sinopsis` con sus respectivos getters y setters.
- `Funcion`: Representa la proyección concreta de una película en una sala y horario dados: `fecha : String`, `horarioHrs : int`, `horarioMin : int`, `peliculaProyectada : Pelicula` y `sala : Sala`. Mantiene su propia lista de `asientos : List<Asiento>` para gestionar la disponibilidad en vivo (`inicializarAsientos()`, `obtenerAsiento()`).
- `Asiento`: Modela cada butaca individual: `identificador : String` (ej. "A1"), `tipoAsiento : String`, `estadoAsiento : String` ("DISPONIBLE", "RESERVADO", "VENDIDO") y `precio : double`. Métodos: `reservar() : boolean` y `vender() : boolean`.

D. Módulo de Venta y Pagos (Patrón Estrategia)

- `Boleto`: Representa el comprobante de venta generado al comprar. Contiene `informacionPelicula : Pelicula`, `informacionSala : String`, `informacionFuncion : Funcion`, `asientosAsignados : List<Asiento>`, `totalPagado : double` y `metodoPago : MetodoDePago`. Métodos: `calcularTotal() : double` e `imprimirBoleto() : void`.
- `MetodoDePago` (Interfaz): Define el contrato polimórfico `procesarPago(monto : double) : boolean`.
  - `TarjetaCredito`: Implementa el pago con `numeroTarjeta : String`.
  - `Paypal`: Implementa el pago con `correo : String`.
  - `TransferenciaElectronica`: Implementa el pago con `cuentaBancaria : String`.

  Ventaja teórica para sustentar: Cumple con el principio **Open/Closed (SOLID)**, permitiendo incorporar nuevas pasarelas de pago sin alterar el código de `Boleto`.

3. Roles de Usuario y Matriz de Permisos (Páginas 4 y 5)

El acceso al sistema está basado en roles mediante la herencia de `Usuario` hacia `Administrador` y `Cliente`:

- **Rol Administrador:** Encargado de operar la programación del cine. Accede mediante su nombre y un PIN numérico de 4 dígitos validado con `validarPin()`. Mantiene el catálogo de películas, la administración de salas y la programación de funciones durante su turno.
- **Rol Cliente:** Usuario consumidor que interactúa con la cartelera para la compra de boletos. Accede al instante sin restricción alguna para consultar disponibilidad de películas, horarios, selección de asientos y pago.

Matriz de Permisos

Acción	Rol Administrador	Rol Cliente
Autenticación con PIN	**SÍ**	**NO**
Alta de Nuevas Películas	**SÍ**	**NO**
Modificación de Películas	**SÍ**	**NO**
Baja de Películas	**SÍ**	**NO**
Gestión de Salas y Asientos	**SÍ**	**NO**
Programación de Funciones	**SÍ**	**NO**
Cancelación de Funciones	**SÍ**	**NO**
Consulta de Cartelera	**SÍ**	**SÍ**
Consulta de Asientos en Función	**SÍ**	**SÍ**
Selección y Compra de Boletos	**NO**	**SÍ**
Procesamiento de Pago Multicanal	**NO**	**SÍ**

4. Diseño del Esquema de Almacenamiento (Base de Datos Relacional)

El esquema relacional cuenta con 8 tablas diseñadas para persistir la estructura de clases respetando integridad referencial en 3ra Forma Normal (3FN):

Tabla 1: cines (Registro de diferentes sucursales)

Campo	Tipo de Dato	Restricción	Descripción
`id_cine`	INT	PRIMARY KEY AUTO_INCREMENT	Identificador único de la sucursal.
`nombre_cine`	VARCHAR(100)	NOT NULL	Nombre comercial de la sucursal (ej. "Cine el Dorado").

Tabla 2: SALAS (Modela las salas dentro de cada cine)

Campo	Tipo de Dato	Restricción	Descripción
`id_sala`	VARCHAR(20)	NOT NULL	Código de la sala ejem: 1
`id_cine`	INT	FOREIGN KEY REFERENCES CINES(id_cine)	Cine al que pertenece la sala.
`filas`	INT	NOT NULL	Número de filas.
`asientos_por_fila`	INT	NOT NULL	Cantidad de asientos por cada fila.
`tipo_sala`	VARCHAR(30)	DEFAULT 'TRADICIONAL'	Formato de proyección (Tradicional, VIP, 3D).
`precio_base`	DECIMAL/FLOAT	NOT NULL	Costo base del boleto para esa sala.
`PK Compuesta`	-	PRIMARY KEY (id_sala, id_cine)	Llave primaria compuesta.

Tabla 3: PELICULAS (Catálogo de películas por día)

Campo	Tipo de Dato	Restricción	Descripción
`id_pelicula`	INT	PRIMARY KEY AUTO_INCREMENT	Identificador único de la película.
`titulo`	VARCHAR(150)	NOT NULL UNIQUE	Título de la película.
`duracion`	VARCHAR(20)	NOT NULL	Duración.
`clasificacion`	VARCHAR(10)	NOT NULL	Restricción de edad.
`genero`	VARCHAR(50)	NOT NULL	Género.
`director`	VARCHAR(100)	NOT NULL	Nombre del director.
`actores`	TEXT	NULL	Lista de actores principales.
`sinopsis`	TEXT	NULL	Resumen de la película.

Tabla 4: FUNCIONES (Fecha de cada película con su sala y horario)

Campo	Tipo de Dato	Restricción	Descripción
`id_funcion`	INT	PRIMARY KEY AUTO_INCREMENT	Identificador único de la proyección.
`fecha`	DATE	NOT NULL	Fecha programada (AAAA-MM-DD).
`horario_hrs`	INT	NOT NULL	Hora de inicio (0 a 23 hrs).
`horario_min`	INT	NOT NULL	Minuto de inicio (0 a 59 min).
`id_pelicula`	INT	FOREIGN KEY REFERENCES PELICULAS(id_pelicula)	Película que se va a proyectar.
`id_sala`	VARCHAR(20)	NOT NULL	Sala donde se proyectará.
`id_cine`	INT	NOT NULL	Sucursal del cine.
`(id_sala, id_cine)`	FK Compuesta	FOREIGN KEY (id_sala, id_cine) REFERENCES SALAS(id_sala, id_cine)	Valida que la sala pertenezca a ese cine.

Tabla 5: ASIENTOS_FUNCION (Control en tiempo real de cada asiento)

Campo	Tipo de Dato	Restricción	Descripción
`id_funcion`	INT	FOREIGN KEY REFERENCES FUNCIONES(id_funcion)	Función a la que corresponde el asiento.
`codigo_asiento`	VARCHAR(10)	NOT NULL	Código de ubicación (ej. "A1", "B4").
`tipo_asiento`	VARCHAR(30)	DEFAULT 'TRADICIONAL'	Tipo de asiento.
`estado`	VARCHAR(20)	DEFAULT 'DISPONIBLE'	Estado actual: 'DISPONIBLE', 'RESERVADO', 'VENDIDO'.
`precio`	DECIMAL(6,2)	NOT NULL	Precio del asiento.
`(id_funcion, codigo_asiento)`	PK Compuesta	PRIMARY KEY (id_funcion, codigo_asiento)	Identificador único del asiento por función.

Tabla 6: USUARIOS (Registro de personal administrativo y clientes)

Campo	Tipo de Dato	Restricción	Descripción
`id_usuario`	INT	PRIMARY KEY AUTO_INCREMENT	Id del usuario.
`nombre`	VARCHAR(100)	NOT NULL	Nombre del usuario.
`rol`	VARCHAR(20)	NOT NULL	Tipo de usuario: admin o usuario.
`pin_acceso`	VARCHAR(10)	NULL	Clave secreta.

Tabla 7: BOLETOS (Historial de transacciones de compra y facturación)

Campo	Tipo de Dato	Restricción	Descripción
`id_boleto`	INT	PRIMARY KEY AUTO_INCREMENT	Ticket de venta.
`id_funcion`	INT	FOREIGN KEY REFERENCES FUNCIONES(id_funcion)	Función adquirida.
`id_usuario`	INT	FOREIGN KEY REFERENCES USUARIOS(id_usuario)	Cliente que realizó la compra.
`total_pagado`	DECIMAL(8,2)	NOT NULL	Monto total cobrado.
`metodo_pago`	VARCHAR(30)	NOT NULL	Pasarela: 'TARJETA_CREDITO', 'PAYPAL', 'TRANSFERENCIA'.
`fecha_compra`	DATETIME	DEFAULT CURRENT_TIMESTAMP	Fecha y hora exacta de la transacción.

Tabla 8: DETALLE_BOLETO_ASIENTOS (Asocia butacas específicas a cada boleto vendido)

Tabla intermedia para asociar qué butacas específicas ampara cada boleto vendido (relación muchos a muchos).

Campo	Tipo de Dato	Restricción	Descripción
`id_boleto`	INT	FOREIGN KEY REFERENCES BOLETOS(id_boleto)	Ticket de compra.
`id_funcion`	INT	NOT NULL	Función a la que pertenecen los asientos.
`codigo_asiento`	VARCHAR(10)	NOT NULL	Asiento adquirido (ej. "A1").
`(id_funcion, codigo_asiento)`	FK Compuesta	FOREIGN KEY (id_funcion, codigo_asiento) REFERENCES ASIENTOS_FUNCION(id_funcion, codigo_asiento)	Valida que el asiento exista en esa función.
`(id_boleto, id_funcion, codigo_asiento)`	PK Compuesta	PRIMARY KEY (id_boleto, id_funcion, codigo_asiento)	Identificador único del detalle.

5. UX / Mockups y Prueba de Desarrollo Visual (Páginas 3, 16 y 17)

- **Ventana de Login (`Sistema Cine - Login`):** Cuenta con dos botones principales para alternar el tipo de visitante (*Administrador* o *Cliente*), un campo para ingresar el PIN numérico y el botón *Ingresar*.
- **Ventana de Cliente (`Sistema - Cliente`):** Incluye el encabezado "*¿Qué vamos a ver hoy?*" con una barra de búsqueda en tiempo real (*buscando...*), una galería cuadrícula de películas, y botones de acción para *revisar horario*, *comprar* y *Cerrar sesión*.
- **Ventana de Administrador (`Sistema - Cine`):** Panel de control con menú lateral izquierdo para seleccionar la entidad a gestionar (*Películas*, *Cines*, *Salas*, *Funciones*) y una tabla central interactiva con columnas *película / valor actual / valor nuevo* (ej. cambiar título de *Kung Fu Panda* a *Iron Man 2*, duración de *1:25* a *1:32*, género y clasificación) con el botón *guardar cambios*.

> **Puntos Clave para la Defensa Oral:**
>
> - **POO:** Aplicación de herencia en `Usuario` , polimorfismo con `MetodoDePago` , encapsulamiento en todas las entidades y arquitectura modular.
> - **Integridad de Datos:** Claves foráneas compuestas que impiden incongruencias entre cines, salas y funciones.
> - **Concurrencia de Asientos:** Desacoplamiento de la butaca física mediante `ASIENTOS_FUNCION` , permitiendo que cada función administre su propia disponibilidad independientemente.